

07 — Orchestration Service: Core Modules

What it is: SAP AI Core's managed pipeline layer — one declarative config runs templating, safety, and the LLM call server-side, instead of you writing that glue code in Python (contrast with sheet 02's proxy, which is just the raw model call).

Client Setup

```
from gen_ai_hub.orchestration.service import OrchestrationService

# Needs an ORCHESTRATION deployment (scenario "orchestration"), not a model deployment.
service = OrchestrationService(api_url=ORCHESTRATION_DEPLOYMENT_URL)
# omit api_url -> SDK auto-discovers the newest RUNNING orchestration deployment
```

Templating Module

```
from gen_ai_hub.orchestration.models.template import Template, TemplateValue
from gen_ai_hub.orchestration.models.message import SystemMessage, UserMessage

template = Template(messages=[
    SystemMessage("You are a concise assistant."),
    UserMessage("{{?text}}"), # {{?name}} = placeholder, filled at call time
])
# few-shot: add UserMessage/AssistantMessage example PAIRS before the final UserMessage
```

LLM Module

```
from gen_ai_hub.orchestration.models.llm import LLM

llm = LLM(name="gpt-4o-mini", version="latest",
          parameters={"temperature": 0, "max_tokens": 500}) # swap "name" to change provider
```

Content Filtering Module

```
from gen_ai_hub.orchestration.models.content_filter import AzureContentFilter
from gen_ai_hub.orchestration.models.filtering import Filtering, InputFiltering, OutputFiltering

strict = AzureContentFilter(hate=0, sexual=0, self_harm=0, violence=0) # 0=block almost all, 6=allow almost all
filtering = Filtering(
    input_filtering=InputFiltering(filters=[strict]), # checked BEFORE the LLM call
    output_filtering=OutputFiltering(filters=[strict]), # checked AFTER the LLM responds
)
```

Data Masking Module

```
from gen_ai_hub.orchestration.models.data_masking import DataMasking
from gen_ai_hub.orchestration.models.sap_data_privacy_integration import (
    SAPDataPrivacyIntegration, MaskingMethod, ProfileEntity)

masking = DataMasking(providers=[SAPDataPrivacyIntegration(
    method=MaskingMethod.PSEUDONYMIZATION, # reversible - real value restored in final response
    entities=[ProfileEntity.PERSON, ProfileEntity.EMAIL],
)])
```

Running It — Config + Service

```
from gen_ai_hub.orchestration.models.config import OrchestrationConfig

config = OrchestrationConfig(template=template, llm=llm, filtering=filtering, data_masking=masking)
result = service.run(config=config, template_values=[TemplateValue(name="text", value="Hi!")])
print(result.orchestration_result.choices[0].message.content)
```

Key Points

- Pipeline **execution order**: masking → input filtering → templating → (grounding) → LLM → output filtering → un-masking.
- **Anonymization** = one-way (generic label, original discarded). **Pseudonymization** = the LLM sees a fake stand-in, but your app gets the real value back automatically.
- Input filtering saves cost/latency by blocking abuse before the LLM runs; output filtering catches unsafe content the model itself generates.
- Every module is **optional and composable** — an `OrchestrationConfig` only needs template
 - `llm`; everything else is additive.

Interview Q&A

Q: Orchestration Service vs. Gen AI Hub proxy — when do you pick which? A: Proxy for a single model call with no extra governance needs; Orchestration when you need masking/filtering/grounding/translation applied consistently and auditable, without writing that logic in every service that calls the LLM.

Q: What do the AzureContentFilter severity numbers (0/2/4/6) mean? A: The highest severity you're willing to ALLOW through per category (hate/violence/self_harm/ sexual) — content scored above your threshold triggers a block.

Q: Why mask PII before the LLM call rather than after? A: So the raw personal data never leaves your system boundary or reaches a third-party model provider's servers/logs — masking must happen before, unmasking happens after (pseudonymization only).